

zkFedMoE — Hinglish Explainer

Zero-Knowledge Federated Mixture-of-Experts
Major Project | Group #34 | IIIT Kota | April 2026

1. Project ek line me kya hai?

zkFedMoE ek **Federated Learning** system hai jo **privacy** (DP-SGD), **verifiability** (SEPG proofs), aur **communication efficiency** (Top-K MoE) — teeno ko ek saath solve karta hai. Matlab: kayi clients (jaise hospitals/phones) apna data share kiye bina ek shared AI model train karte hain, aur server proof check karke maan leta hai ki sab kuch sahi se hua.

Real-world analogy

Socho 5 hospitals hain. Sab apne marizon ka data share nahi kar sakte (privacy law). Lekin sab ek hi diagnostic AI ko milke better banana chahte hain. zkFedMoE bolta hai — koi data bhej hi mat, sirf *model updates* bhejo, aur unme bhi noise daal do (DP), aur ek SHA-256 proof bhejo ki tumne sahi rules follow kiye. Server proof verify karke updates ko average kar deta hai.

2. App ko start kaise karein?

Step 1 — Dependencies install (one-time)

```
cd "d:/projects/Major Project/Code"
pip install torch pandas numpy streamlit plotly matplotlib graphviz reportlab
```

Step 2 — Tin tarike se chalao (apni zarurat ke hisaab se)

Tarika	Command	Kab use karein
Dashboard (recommended)	python -m streamlit run dashboard.py	Viva, demo, ya interactive exploration
Full experiments	python -m experiments.run_all_experiments	Saare 4 experiments ki plots banane ke liye (~10-15 min CPU)
Quick demo	python run_demo.py	Phase 1+2 ka end-to-end ek baar mein dekhne ke liye (~3 min)

Dashboard browser mein `http://localhost:8501` par khulta hai. 10 pages hain — Home se start karke About tak.

3. Pura system kaise kaam karta hai? (One Round)

Federated Learning mein ek 'round' matlab ek poora cycle. Hamare project mein 5 rounds default hain. Har round mein 8 steps:

- **Server** apna global model (θ_t) sab clients ko bhejta hai.
- **Client** apne private data par local training karta hai (data kabhi nahi nikalta).
- Top-K experts select karta hai — sab experts ki jagah sirf K most-used.
- **DP-SGD** apply: gradient clip karo (max norm C), Gaussian noise daalo (sigma).
- **SEPG proof** banao: TopK list + clip_norm + sigma + SHA-256 hash.
- Sparse update + proof server ko bhejo (sirf bytes, data nahi).
- **Server** 4-check verify karta hai — fail hua to client reject.
- Sab pass hue updates ko aggregate karo (FedAvg / Median / TrimMean) $\Rightarrow \theta_{t+1}$.

Key insight: Raw data network par kabhi travel nahi karta. Sirf model gradients aur ek chhota proof. Yahi privacy-preserving FL ka core hai.

4. Saari functionalities — kya, kyu, kahan

#	Feature	Kya karta hai (Hinglish)	File
1	MoE + LoRA model	Tiny model hai (~600K params). MoE ke 8 experts hain, har input ke liye router decide karta hai konse 2 experts use karne hain (Top-2). LoRA classifier head full Linear ki jagah low-rank A,B matrices use karta hai — trainable params 100x kam.	src/models/moe_model.py
2	AG News Dataset	120K news headlines, 4 classes (World, Sports, Business, Tech). 5 simulated clients me split. Optionally Dirichlet(alpha) split for non-IID setting (real FL jaise).	src/data/text_dataset.py
3	FedAvg client local training	Har client apne shard par 2 epochs train karta hai. Adam optimizer, cross-entropy loss. FedProx $\mu > 0$ set kar do to non-IID drift kam.	src/fl/client.py
4	Top-K Sparse Communication	Client sirf Top-K experts ke weights bhejta hai, baaki ke nahi. $K=2$ of $E=8 \Rightarrow \sim 75\%$ bandwidth saving. Saving = $1 - K/E$.	src/fl/client.py
5	Differential Privacy (DP-SGD)	clip_update() $\rightarrow$ gradient L2 norm ko C par cap karta hai. add_noise() $\rightarrow$ Gaussian $N(0, \sigma^2)$ daalta hai. Renyi accountant (alpha, sigma, q, T) se tight (epsilon, delta) bound deta hai.	src/fl/dp.py
6	SEPG Proof + 4-check Verify	Client banata hai: TopK list, $\ grad\ $, sigma, SHA-256(grad). Server check karta hai: (1) $ TopK =K$, (2) clip_norm $\leq$ max, (3) sigma $\geq$ sigma_min, (4) hash match. Fail = reject client.	src/fl/sepg.py
7	Secure Aggregation (pairwise masks)	Bonawitz et al. 2017 ka simplified version. Har client apni update mein deterministic masks add karta hai jo sum karne par cancel ho jaate hain. Server ko sum dikhta hai, individual update nahi.	src/fl/server.py (aggregate_secure)

8	Adversary Simulations	3 attacks: Poisoning (labels flip), Free-rider (jhoothi training, sirf noise), Sybil (1 attacker, N fake identities). Real FL deployment mein ye sab khatre real hain.	src/fl/adversaries.py
9	Robust Aggregation	FedAvg easy hai par poisoning ke against weak. Coordinate-wise Median = har parameter ka median, byzantine-robust. Trimmed Mean = top/bottom k% drop kar ke average. 40% malicious par Median best (only -7pp drop).	src/fl/server.py
10	Verifiable Oracle Committee	M oracle nodes har client ki update ko independent score karte hain. Final decision = median score (M/2 oracles galat ho to bhi safe). Blockchain-free version of threshold signatures.	src/fl/sepg.py (committee_decision)
11	4 Automated Experiments	Exp1: Privacy-utility tradeoff (sigma vs accuracy). Exp2: Communication savings vs K. Exp3: SEPG verification overhead (~6ms). Exp4: Robustness under 0/20/40% poisoning.	experiments/run_all_experiments.py
12	10-page Streamlit Dashboard	Home, Predict (live text classify), Train (configurable FL), Custom CSV (upload your data), Privacy & DP (live epsilon graph), Robustness, Experiments, Compare, Architecture, About.	dashboard.py

5. Dashboard ke 10 pages — har page kya karta hai

- **Home** — Pipeline ka graphviz diagram, concept cards, demo walkthrough. Yahi se start karna hai.
- **Predict** — Apna text type karo (e.g. "Apple unveils new iPhone") → model live classify karega + dikhayega konse 2 experts route hue.
- **Train** — Configurable FL training. Rounds, clients, K, DP on/off — sab slider/toggles. Live charts.
- **Custom CSV** — Apni CSV upload karo (label, text columns). FL train hoga, confusion matrix dikhega.
- **Privacy & DP** — DP-SGD ke saath training. Live (epsilon, delta) curve. SEPG proof JSON dikhayega.
- **Robustness** — Poisoning/Freerider/Sybil attack simulate karo. FedAvg vs Median vs TrimMean compare hoga.
- **Experiments** — Saare 4 pre-computed experiments ke interactive Plotly charts.
- **Compare** — Real-time communication savings calculator (slider K).
- **Architecture** — Model data-flow, parameter pie chart, 5 code-snippet tabs.
- **About** — Team, advisor, institution, status table.

6. Math sirf zarurat ke (cheat-sheet for viva)

FedAvg

$\theta_{t+1} = \sum_i (n_i / n) * \theta_{i,t}$

Simply: clients ke models ka weighted average, weight = data size.

Top-K MoE

$y = \text{sum over Top-K experts of } g_k(x) * E_k(x)$

Router 8 experts mein se 2 chunta hai, baaki compute mat karo.

LoRA

$W' = W + B * A$, where $B \in \mathbb{R}^{(d \times r)}$, $A \in \mathbb{R}^{(r \times d)}$, $r \ll d$

Full W ki jagah chote A,B train karo. Param count 100x kam.

DP-SGD: clip + noise

$\bar{g} = g / \max(1, ||g||/C)$ — gradient ko cap karo

$\tilde{g} = (1/B) * (\sum \bar{g} + N(0, \sigma^2 * C^2 * I))$ — noise daalo

(epsilon, delta)-DP

$\Pr[M(D) \in S] \leq e^{\epsilon} * \Pr[M(D') \in S] + \delta$

D, D' me 1 record ka difference. Smaller epsilon = stronger privacy.

Renyi Accountant

$\epsilon(\alpha) = (\alpha / (2 * \sigma^2)) * T * q^2$

T rounds aur sampling rate q ke saath cumulative privacy loss.

Comm Saving

Saving = $1 - K/E$. $K=2, E=8 \Rightarrow 75\%$.

Coordinate-wise Median

$\theta_{t+1}[j] = \text{median}(\theta_{1,j}, \dots, \theta_{N,j})$

Per-parameter median $\Rightarrow$ outlier (poisoned) clients ka effect 0.

7. Results — kya prove hua

Experiment	Result	Interpretation
1. Privacy-Utility	no DP & $\approx 58\%$, $\sigma=0.1$ & $\approx 25\%$	DP correctly accounted; small-scale (5 rounds) accuracy cliff expected
2. Comm Savings (K)	K=4 best (59% acc, 23% saving)	Sweet spot K=3-4. K=8 over-fits (no implicit reg)
3. SEPG Overhead	~ 6 ms/client, K-independent	Negligible vs training time

4. Robustness (40% mal)	FedAvg -16pp, Median -7pp, TrimMed -5pp	Median is most byzantine-robust
-------------------------	---	---------------------------------

8. Har piece kyu zaruri hai? (Viva ka Q&A)

Q. FL hi kyu, central training kyu nahi?

Privacy laws (GDPR, HIPAA) raw data centralize karne nahi dete. FL me data device se nikalti hi nahi.

Q. MoE kyu use kiya, simple FFN kyu nahi?

MoE ke saath har client ko sirf K experts share karne padte hain, saare nahi $\Rightarrow$ communication $O(N)$ se $O(K)$ ho jaata hai. Ye paper ka USP hai.

Q. LoRA ka kya role hai?

Trainable parameters drastically reduce karta hai. Edge devices (phones) par memory aur compute kam hai — LoRA fits.

Q. DP-SGD me clip pehle kyu, noise pehle kyu nahi?

Clip pehle kyunki gradient sensitivity bound karna hai. Sensitivity = C ho gayi, ab noise $\sigma \cdot C$ add karenge $\Rightarrow$ (epsilon, delta) guarantee.

Q. SEPG zero-knowledge kaise hai?

Hum cryptographic ZK-SNARK use nahi karte (heavy hai). Hum SHA-256 commitment + structural checks karte hain — lightweight ZK-style verification. Server gradient nahi dekh sakta, sirf properties check karta hai.

Q. Median FedAvg se kyu better under attack?

FedAvg outliers se kheech jaata hai (mean is sensitive). Median byzantine-robust hai — up to $N/2$ malicious clients tak unaffected.

Q. Hum 100 patients use karte hain kya?

Nahi. Hum AG News (120K news headlines, 4 classes) use karte hain as a public benchmark. Healthcare to motivating example hai — techniques same hi rahengi.

9. File-by-file map

File / Folder	Kya hai
Code/dashboard.py	Streamlit 10-page dashboard (~2,061 lines). UI entry point.
Code/run_demo.py	Phase 1 + Phase 2 ek shot mein. Plots banata hai.
Code/src/models/moe_model.py	LoRA, MoEEExpert, MoELayer, MoETextClassifier.
Code/src/data/text_datasets.py	AG News loader, vocab build, IID + Dirichlet split.
Code/src/fl/client.py	local_train(): forward+backward, FedProx option, sparse pack.
Code/src/fl/server.py	FedServer: aggregate, aggregate_median, aggregate_trimmed_mean, aggregate_secure, aggregate_with_verification.
Code/src/fl/dp.py	clip_update, add_noise, apply_dp, PrivacyAccountant, RenyiAccountant.
Code/src/fl/sepg.py	SEPGProof, generate_proof, verify_proof, oracle_score, committee_decision.
Code/src/fl/adversaries.py	poisoning_train, freerider_train, sybil_clones.
Code/experiments/run_all_experiments.py	4 experiments → PNGs + JSON in plots/.
Code/data/ag_news_train.csv	120,000 train samples (label, text).
Code/data/ag_news_test.csv	7,600 test samples.
Code/plots/	Pre-computed PNGs + experiment_results.json.
Report/main.tex	Final report LaTeX (46 references, full methodology).

10. Team

Group #34 — IIIT Kota

- Keshav Kashyap (2023KUCP1161)
- Lakshya Sharma (2023KUCP1167)
- Prakriti Patel (2023KUCP1109)

Supervisor: Dr. Gyan Singh Yadav, CSE Dept., IIIT Kota

Quick recap (1 line each)

- **Kya hai:** Privacy + Verifiable + Communication-efficient Federated Learning.
- **Data:** AG News, 120K headlines, 4 classes, 5 simulated clients.
- **Privacy:** DP-SGD (clip + Gaussian noise) + Renyi accountant.
- **Verifiability:** SEPG proof — SHA-256 + 4 structural checks.
- **Comm savings:** Top-K MoE ⇒ ~75% bandwidth reduction.
- **Robustness:** Median agg ⇒ -7pp drop at 40% malicious (vs FedAvg -16pp).

- **Run:** `python -m streamlit run dashboard.py` ⇒ localhost:8501.

— END —